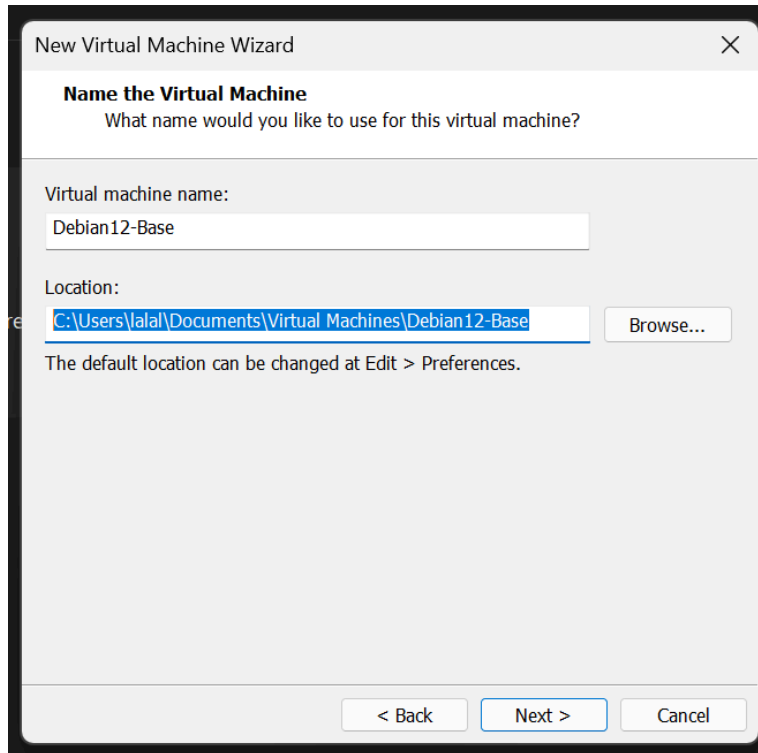


Projet Terraform

Je débute le projet en créant ma première Machine Virtuelle, que je nomme "Debian12-Base". Je vais garder et prendre en capture d'écran le chemin de la VM, car on en aura besoin plus tard.



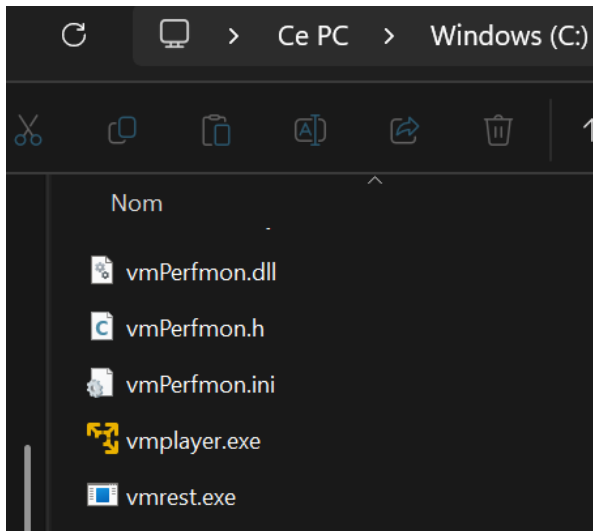
Après ça, je vais installer Terraform.

Je vais sur le site developer.hashicorp.com/terraform/install, et je sélectionne la version Windows AMD64 que je vais télécharger sur mon pc. Je vais dans mes variables d'environnement pour y faire un nouveau chemin, ou je vais coller mon chemin Terraform. C:\Users\lala\Documents\Terraform

```
PS C:\Users\lala> terraform -version
Terraform v1.15.8
on windows_amd64
PS C:\Users\lala>
```

Et on est bons ! Terraform a bien été installé !

Je vais pouvoir passer à la configuration de vmrest.exe, il est normalement déjà installé avec VMWare Workstation Pro !



```
PS C:\WINDOWS\system32> cd "C:\Program Files (x86)\VMware\VMware Workstation"
PS C:\Program Files (x86)\VMware\VMware Workstation> .\vmrest.exe --config
VMware Workstation REST API
Copyright (C) 2018-2025 Broadcom.
All Rights Reserved

vmrest 1.3.1 build-24583834
Username:root
New password:
Password does not meet complexity requirements:
- Minimum 1 uppercase character
- Minimum 1 lowercase character
- Minimum 1 numeric digit
- Minimum 1 special character(!#$%&'()*+,-./:;<=>?@[ ]^_`{|}~)
- Length between 8 and 12
New password:
Retype new password:
Processing...
Credential updated successfully
PS C:\Program Files (x86)\VMware\VMware Workstation>
```

Je fais mon nouveau compte via Powershell en admin, et je choisis mon nom d'utilisateur ainsi que mon mot de passe.

```
PS C:\Program Files (x86)\VMware\VMware Workstation> .\vmrest.exe
VMware Workstation REST API
Copyright (C) 2018-2025 Broadcom.
All Rights Reserved

vmrest 1.3.1 build-24583834
-
Using the VMware Workstation UI while API calls are in progress is not recommended and may yield unexpected results.
-
Serving HTTP on 127.0.0.1:8697
-
Press Ctrl+C to stop.
```

Ca tourne bien ! On voit que ça écoute sur 127.0.0.1:8697 !

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\lala> Invoke-WebRequest -Uri "http://127.0.0.1:8697/api/vms"
Invoke-WebRequest : { "Code": 1, "Message": "Authentication failed" }
Au caractère Ligne:1 : 1
+ Invoke-WebRequest -Uri "http://127.0.0.1:8697/api/vms"
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebEx
ception
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
PS C:\Users\lala>
```

C'est ce qu'on voulait, car le message "Authentication Failed" montre bien que le serveur tourne, il attend juste qu'on s'identifie.

```
StatusCode      : 200
StatusDescription : OK
Content         : {91, 10, 32, 32...}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 562
                  Cache-Control: no-cache
                  Content-Type: application/vnd.vmware.vmw.rest-v1+json
                  Date: Mon, 27 Jul 2026 10:57:08 GMT

                  [
                    {
                      "id": "MER67SOPF38U5B1V3AREI0D4UN...
Headers         : {[Content-Length, 562], [Cache-Control, no-cache], [Content-Type,
                  application/vnd.vmware.vmw.rest-v1+json], [Date, Mon, 27 Jul 2026 10:57:08 GMT]}
RawContentLength : 562
```

Le code 200 montre bien que c'est un succès, mais il y a un petit problème d'affichage que je vais régler en montrant l'entièreté l'id et le path.

```
PS C:\Users\lala> Invoke-RestMethod -Uri "http://127.0.0.1:8697/api/vms" -Headers @{Authorization="Basic $cred"} | ConvertTo-Json
{
  "value": [
    {
      "id": "MER67SOPF38U5B1V3AREI0D4UNGI01GR",
      "path": "C:\\Users\\lala\\Documents\\Virtual Machines\\Debian12-Base\\Debian12-Base.vmx"
    },
    {
      "id": "GDIA96IOHDB32H70BISAK1G864AGQC6G",
      "path": "C:\\Users\\lala\\Documents\\Virtual Machines\\ansible-ctrl\\ansible-ctrl.vmx"
    },
    {
      "id": "2BCBE282UIELF3MBI371ETE9LOQD7NH6",
      "path": "C:\\Users\\lala\\Documents\\Virtual Machines\\db01\\db01.vmx"
    },
    {
      "id": "TTFI3USA0G0261QBQ6UK38N4MLFCGHCE",
      "path": "C:\\Users\\lala\\Documents\\Virtual Machines\\web01\\web01.vmx"
    }
  ],
  "Count": 4
}
PS C:\Users\lala>
```

On voit bien ma VM et les autres dans mon chemin d'accès. Je vais mettre de côté (Feldup) l'idd de ma VM pour m'en servir tout à l'heure.

MER67SOPF38U5B1V3AREI0D4UNGI01GR

Je vais maintenant créer le dossier de mon projet Terraform.

```
main.tf  X  +
Fichier  Modifier  Affichage

terraform {
  required_providers {
    vmworkstation = {
      source = "elsudano/vmworkstation"
      version = "1.0.4"
    }
  }
}

provider "vmworkstation" {
  user      = "root"
  password  = "Alix2008*"
  url       = "http://127.0.0.1:8697/api"
  https     = false
  debug     = "none"
}

resource "vmworkstation_vm" "test_vm" {
  sourceid      = "MER67SOPF38U5B1V3AREI0D4UNGI01GR"
  denomination  = "test-vm"
  description   = "Premier test de clonage via Terraform"
  path          = "C:\\Users\\lala1\\Documents\\Virtual Machines\\test-vm\\test-vm.vmx"
  processors    = 2
  memory       = 2048
}
```

Je vais l'appeler [main.tf](#), comme convenu dans la consigne.

```
PS C:\Users\lala1\terraform_debian_lab> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding elsudano/vmworkstation versions matching "1.0.4"...
- Installing elsudano/vmworkstation v1.0.4...
- Installed elsudano/vmworkstation v1.0.4 (self-signed, key ID 25B4C7DDA0B6F683)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\lala1\terraform_debian_lab> |
```

Youpi, tralala, tsoin tsoin. Tout fonctionne, youpi. Je n'en peux plus de ce projet.

```

PS C:\Users\lala\terraform_debian_lab> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# vmworkstation_vm.test_vm will be created
+ resource "vmworkstation_vm" "test_vm" {
  + denomination = "test-vm"
  + description  = "Premier test de clonage via Terraform"
  + id          = (known after apply)
  + memory      = 2048
  + path        = "C:\\Users\\lala\\Documents\\Virtual Machines\\test-vm\\test-vm.vmx"
  + processors   = 2
  + sourceid    = "MER67SOPF38U5B1V3AREI0D4UNGI01GR"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
PS C:\Users\lala\terraform_debian_lab>

```

Et voilà, ça fonctionne comme on le voulait. Je fais toujours correctement ce qu'on me demande pour avoir des mauvaises notes à la fin. Bon.

```

Terraform will perform the following actions:

# vmworkstation_vm.test_vm will be created
+ resource "vmworkstation_vm" "test_vm" {
  + denomination = "test-vm"
  + description  = "Premier test de clonage via Terraform"
  + id          = (known after apply)
  + memory      = 2048
  + path        = "C:\\Users\\lala\\Documents\\Virtual Machines\\test-vm\\test-vm.vmx"
  + processors   = 2
  + sourceid    = "MER67SOPF38U5B1V3AREI0D4UNGI01GR"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

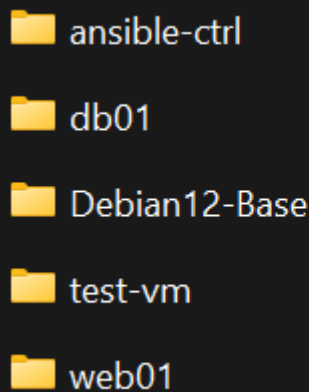
Enter a value: yes

vmworkstation_vm.test_vm: Creating...
vmworkstation_vm.test_vm: Still creating... [00m10s elapsed]
vmworkstation_vm.test_vm: Creation complete after 14s [id=U4L326P66V37GAU5HCP1EB0JLOR7MQI1]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\lala\terraform_debian_lab>

```

CA MARCHE ! Ca roule, ça court ! J'adore.



Ma VM “test-vm” est bien créée ! Je suis la plus heureuse. J’adore mon métier. Maintenant, je vais faire mes deux autres VM. J’ai supprimé la VM test juste en tapant “terraform destroy”.

```
terraform {
  required_providers {
    vmworkstation = {
      source = "elsudano/vmworkstation"
      version = "1.0.4"
    }
  }
}

provider "vmworkstation" {
  user      = "root"
  password  = "Alix2008*"
  url       = "http://127.0.0.1:8697/api"
  https     = false
  debug     = false
}

resource "vmworkstation_vm" "kali_like" {
  sourceid      = "MER67SOPF38U5B1V3AREI0D4UNGIO1GR"
  denomination  = "kali-like-vm"
  description   = "VM attaquante"
  path          = "C:\\Users\\l1l1l1\\Documents\\Virtual Machines\\kali-like-vm\\kali-like-vm.vmx"
  processors    = 2
  memory       = 2048
}

resource "vmworkstation_vm" "victim" {
  sourceid      = "MER67SOPF38U5B1V3AREI0D4UNGIO1GR"
  denomination  = "victim-vm"
  description   = "VM victime"
  path          = "C:\\Users\\l1l1l1\\Documents\\Virtual Machines\\victim-vm\\victim-vm.vmx"
  processors    = 2
  memory       = 2048
}
```

Plan: 2 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

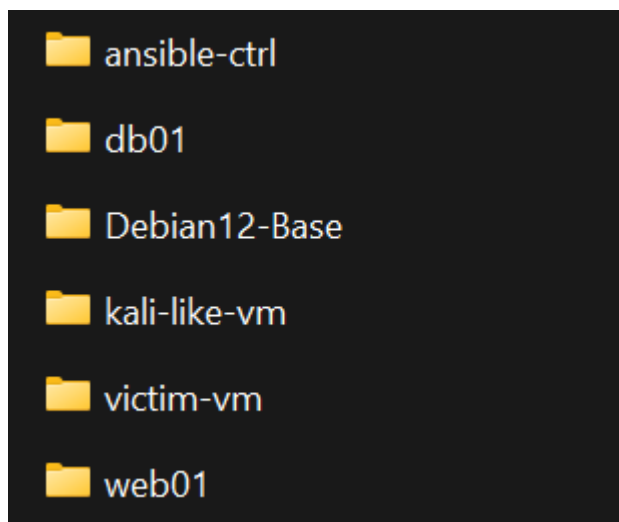
PS C:\Users\l1l1l1\terraform> terraform destroy

Les deux VMs ont été créées, comme on peut le voir avec le “Plan: 2 to add”.

```
vmworkstation_vm.victim: Creating...  
vmworkstation_vm.victim: Still creating... [00m10s elapsed]  
vmworkstation_vm.victim: Creation complete after 15s [id=59TLE58MELTTL3BDMR3FDMQ1QRL9JAEJ]  
vmworkstation_vm.kali_like: Modifying... [id=QGFAB6PDLF5P09TVICA5VPJA14KMIPM7]  
vmworkstation_vm.kali_like: Modifications complete after 0s [id=QGFAB6PDLF5P09TVICA5VPJA14KMIPM7]  
  
Apply complete! Resources: 1 added, 1 changed, 0 destroyed.  
PS C:\Users\lala\terraform_debian_lab> |
```

```
vmworkstation_vm.victim: Creating...  
vmworkstation_vm.kali_like: Creating...  
vmworkstation_vm.kali_like: Still creating... [00m10s elapsed]  
vmworkstation_vm.kali_like: Creation complete after 14s [id=QGFAB6PDLF5P09TVICA5VPJA14KMIPM7]
```

Et les deux VMs sont bien créées ! On peut le vérifier dans les documents de mes Vms, et effectivement elles sont bien là.



Pendant mon projet, j’ai rencontré un problème ; mes trois VMs n’affichaient rien, même lorsque je les relançais. J’ai donc décidé de les abandonner pour cloner sur une seule VM qui fonctionnait correctement, en changeant le hostname.

Dans mon main.tf, je vais donc remplacer le sourceid par ma nouvelle ID. Je n’ai pas besoin de remplacer l’id sur la VM de base - la première que j’ai faite - même si elle ne fonctionne pas, car je n’en aurai plus besoin pour la suite du projet.

```

resource "vmworkstation_vm" "kali_like" {
  sourceid = "KCD23RV9ASIOBBIJ3NA2N9GBE8KMALM2"
  denomination = "kali-like-vm"
  description = "VM attaquante"
  path = "C:\\Users\\lala\\Documents\\Virtual Machines\\kali-like-vm\\kali-like-vm.vmx"
  processors = 2
  memory = 2048
}

resource "vmworkstation_vm" "victim" {
  sourceid = "KCD23RV9ASIOBBIJ3NA2N9GBE8KMALM2"
  denomination = "victim-vm"
  description = "VM victime"
  path = "C:\\Users\\lala\\Documents\\Virtual Machines\\victim-vm\\victim-vm.vmx"
  processors = 2
  memory = 2048
}

```

Je retire mes précédentes VMs, sinon les nouvelles ne voudront pas se créer :

```

PS C:\Users\lala\terraform_debian_lab> Remove-Item -Recurse -Force "C:\Users\lala\Documents\Virtual Machines\kali-like-vm"
PS C:\Users\lala\terraform_debian_lab> Remove-Item -Recurse -Force "C:\Users\lala\Documents\Virtual Machines\victim-vm"
PS C:\Users\lala\terraform_debian_lab> |

```

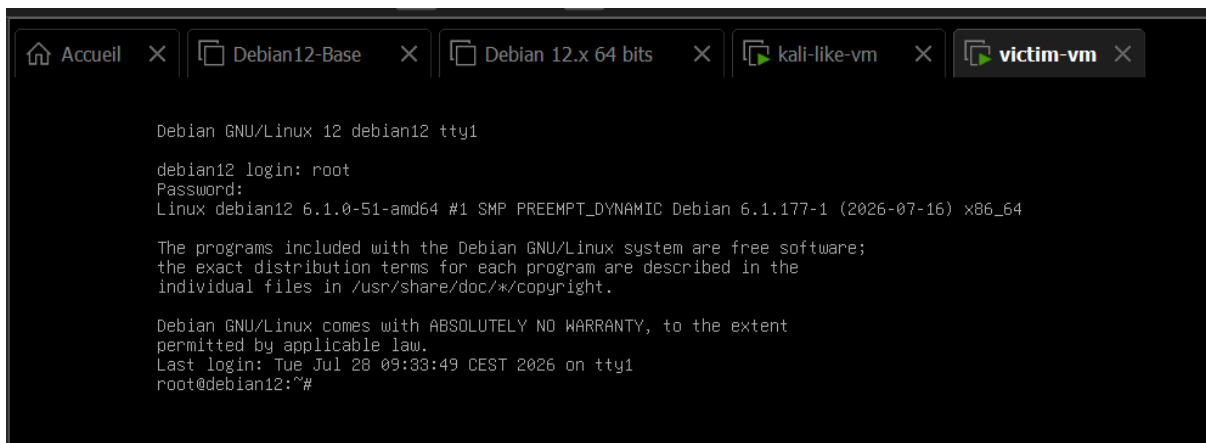
```

vmworkstation_vm.victim: Creating...
vmworkstation_vm.victim: Still creating... [00m10s elapsed]
vmworkstation_vm.victim: Creation complete after 16s [id=59TLE58MELTTL3BDMR3FDMQ1QRL9JAEJ]
vmworkstation_vm.kali_like: Creating...
vmworkstation_vm.kali_like: Still creating... [00m10s elapsed]
vmworkstation_vm.kali_like: Creation complete after 14s [id=QGFA6PDLF5P09TVICA5VPJA14KMIPM7]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\Users\lala\terraform_debian_lab>

```

Cette fois, les deux Vms sont créées, et celles-là sont à partir de la bonne template.



```

Debian GNU/Linux 12 debian12 tty1

debian12 login: root
Password:
Linux debian12 6.1.0-51-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.177-1 (2026-07-16) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Jul 28 09:33:49 CEST 2026 on tty1
root@debian12:~#

```


Maintenant, on peut crier victoire ! les deux Vms tournent correctement. Je vais pouvoir changer le hostname sur chaque VM, sinon les deux auront le même nom réseau, ce qui peut causer des soucis... encore.

```
GNU nano 7.2
127.0.0.1    localhost
127.0.1.1    kali-like-vm_

# The following lines are desirable for IPv6 capable hosts
::1          localhost ip6-localhost ip6-loopback
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters
```

Inutile de montrer sur l'autre VM, puisque c'est exactement le même procédé.

```
root@debian12:~# ping -c 4 192.168.88.130
PING 192.168.88.130 (192.168.88.130) 56(84) bytes of data.
64 bytes from 192.168.88.130: icmp_seq=1 ttl=64 time=1.37 ms
64 bytes from 192.168.88.130: icmp_seq=2 ttl=64 time=1.33 ms
64 bytes from 192.168.88.130: icmp_seq=3 ttl=64 time=0.826 ms
64 bytes from 192.168.88.130: icmp_seq=4 ttl=64 time=0.537 ms

--- 192.168.88.130 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.537/1.015/1.370/0.349 ms
root@debian12:~#
```

Je ping mon autre VM à partir de kali, et c'est bon, les VMs se voient bien sur le réseau ! Après avoir perdu trois heures à régler des problèmes à cause de ce satané projet, je vais enfin pouvoir travailler. J'installe masscan ainsi que nmap sur ma VM kali :

```
root@kali-like-vm:~# nmap --version
Nmap version 7.93 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.3.6 openssl-3.0.20 libssh2-1.10.0 libz-1
.2.13 libpcap-1.10.3 nmap-libdnet-1.12 ipv6
Compiled without:
Available nsock engines: epoll poll select
root@kali-like-vm:~# masscan --version

Masscan version 1.3.2 ( https://github.com/robertdavidgraham/mas
scan )
Compiled on: Feb  2 2021 11:13:27
Compiler: gcc 10.2.1 20210110
OS: Linux
CPU: unknown (64 bits)
GIT version: unknown
root@kali-like-vm:~#
```

Sur ma pauvre VM victime, je fais tourner le serveur web.

```
root@victim-vm:/tmp/vulnerable-web# wget -q0- http://localhost:8080
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
</ul>
<hr>
</body>
</html>
root@victim-vm:/tmp/vulnerable-web#
```

On va pouvoir passer en scénario d'attaque complet. Y'a carrément plein de guerres dans le monde, je vois pas pourquoi on fait un truc pareil.

```
root@kali-like-vm:~# nmap -p 8080 192.168.88.130
Starting Nmap 7.93 ( https://nmap.org ) at 2026-07-28 10:36 CEST
Nmap scan report for 192.168.88.130
Host is up (0.0014s latency).

PORT      STATE SERVICE
8080/tcp  open  http-proxy
MAC Address: 00:0C:29:CA:3F:1A (VMware)

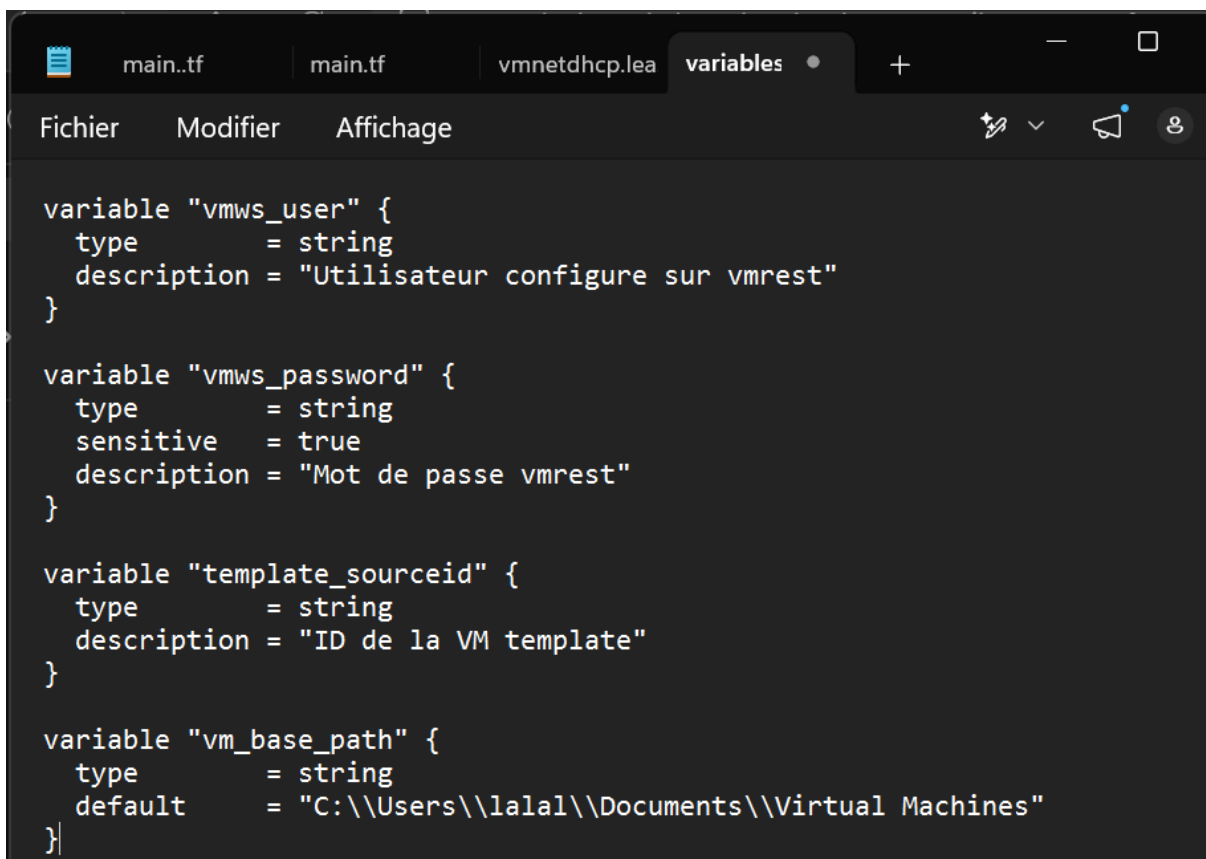
Nmap done: 1 IP address (1 host up) scanned in 0.30 seconds
root@kali-like-vm:~#
```

Et voilà ! J'ai du sang sur les mains. kali peut scanner victim-vm avec nmap, elle détecte bien le port 8080 ouvert, avec le service HTTP dessus, et les deux VMs communiquent sur le même réseau virtuel. Vous désirez en voir davantage ? Je vais vous régaler.

```
root@kali-like-vm:~# curl http://192.168.88.130:8080
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
</ul>
<hr>
</body>
</html>
root@kali-like-vm:~#
```

Ça confirme mon hostilité : l'attaquante peut scanner et interagir avec le service vulnérable de la victime.

On va pouvoir enchaîner sur les variables !

A screenshot of a code editor with a dark theme. The editor has several tabs at the top: 'main..tf', 'main.tf', 'vmnetdhcp.lea', and 'variables'. The 'variables' tab is currently selected. Below the tabs, there is a menu bar with 'Fichier', 'Modifier', and 'Affichage'. The main area of the editor contains Terraform variable definitions. The variables are: 'vmws_user' (string, description: 'Utilisateur configure sur vmrest'), 'vmws_password' (string, sensitive: true, description: 'Mot de passe vmrest'), 'template_sourceid' (string, description: 'ID de la VM template'), and 'vm_base_path' (string, default: 'C:\\\\Users\\\\lallal\\\\Documents\\\\Virtual Machines').

```
variable "vmws_user" {
  type      = string
  description = "Utilisateur configure sur vmrest"
}

variable "vmws_password" {
  type      = string
  sensitive  = true
  description = "Mot de passe vmrest"
}

variable "template_sourceid" {
  type      = string
  description = "ID de la VM template"
}

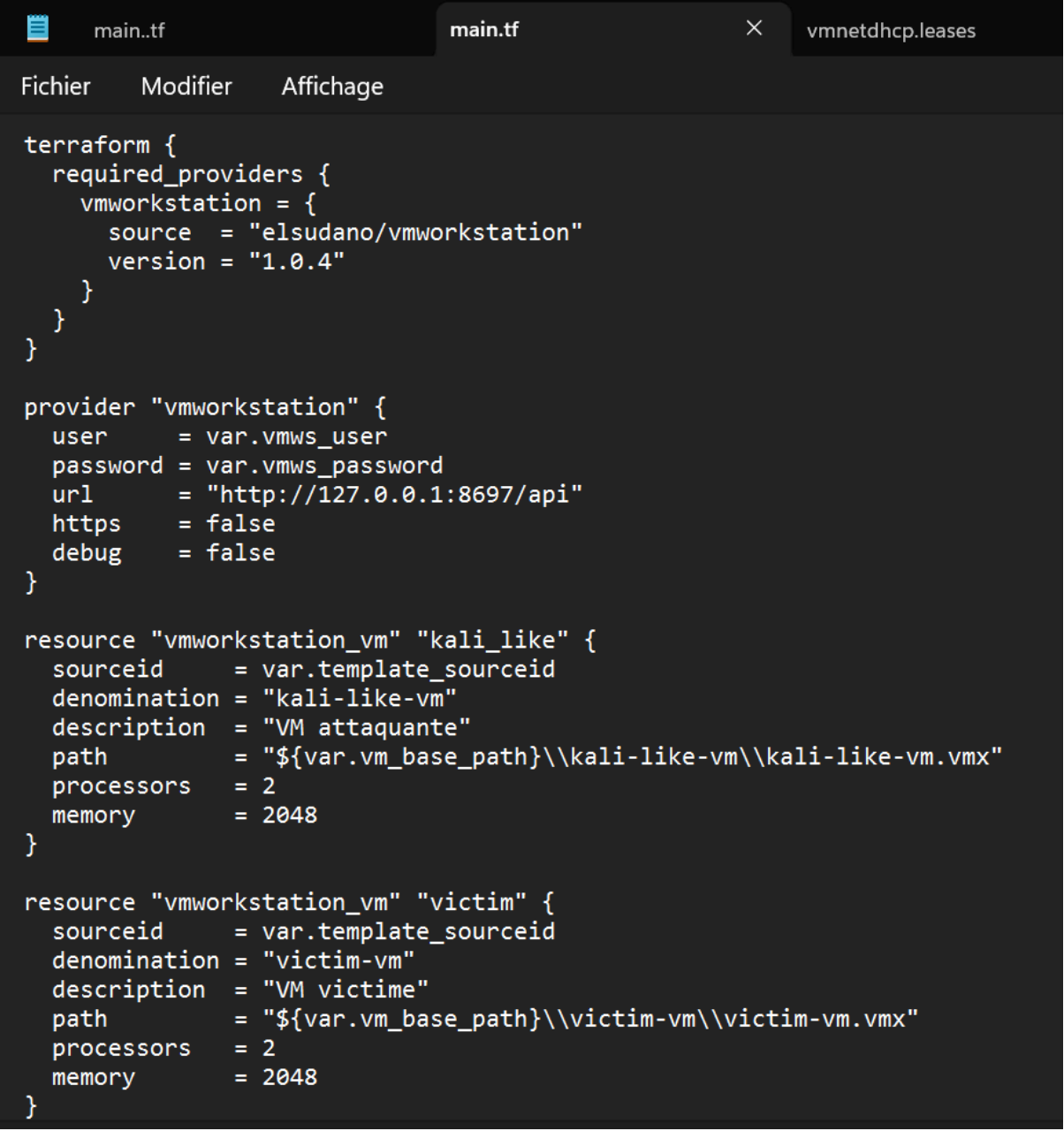
variable "vm_base_path" {
  type      = string
  default    = "C:\\\\Users\\\\lallal\\\\Documents\\\\Virtual Machines"
}
```

Je fais un nouveau fichier que je vais appeler "[variables.tf](#)", et j'y écris ce que je viens de vous montrer, pas ma commande de pizza.

```
vmws_user      = "root"
vmws_password  = "Alix2008*"
template_sourceid = "KCD23RV9ASIOBBIJ3NA2N9GBE8KMALM2"
vm_base_path   = "C:\\Users\\lala\\Documents\\Virtual Machines"
```

Je fais maintenant un fichier que je nomme "terraform.tfvars", et j'y colle ceci avec mes informations vmws.

Puis, je remplace tout mon fichier [main.tf](#) par ceci :



```
main.tf main.tf vmnetdhcp leases
Fichier Modifier Affichage

terraform {
  required_providers {
    vmworkstation = {
      source = "elsudano/vmworkstation"
      version = "1.0.4"
    }
  }
}

provider "vmworkstation" {
  user      = var.vmws_user
  password  = var.vmws_password
  url       = "http://127.0.0.1:8697/api"
  https     = false
  debug     = false
}

resource "vmworkstation_vm" "kali_like" {
  sourceid      = var.template_sourceid
  denomination   = "kali-like-vm"
  description    = "VM attaquante"
  path           = "${var.vm_base_path}\\kali-like-vm\\kali-like-vm.vmx"
  processors     = 2
  memory        = 2048
}

resource "vmworkstation_vm" "victim" {
  sourceid      = var.template_sourceid
  denomination   = "victim-vm"
  description    = "VM victime"
  path           = "${var.vm_base_path}\\victim-vm\\victim-vm.vmx"
  processors     = 2
  memory        = 2048
}
```

Ce qui change concrètement, c'est que `var.vmws_user`, `var.vmws_password`, `var.template_sourceid` et `var.vm_base_path` viennent piocher mes valeurs dans mon nouveau `terraform.tfvars`, au lieu d'être écrits directement ici.

```

PS C:\Users\lala\terraform_debian_lab> terraform plan
vmworkstation_vm.victim: Refreshing state... [id=59TLE58MELTTL3BDMR3FDMQ1QRL9JAEJ]
vmworkstation_vm.kali_like: Refreshing state... [id=QGFAB6PDLF5P09TVICA5VPJA14KMIPM7]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

# vmworkstation_vm.kali_like will be updated in-place
~ resource "vmworkstation_vm" "kali_like" {
  + description = "VM attaquante"
    id          = "QGFAB6PDLF5P09TVICA5VPJA14KMIPM7"
    # (5 unchanged attributes hidden)
}

# vmworkstation_vm.victim will be updated in-place
~ resource "vmworkstation_vm" "victim" {
  + description = "VM victime"
    id          = "59TLE58MELTTL3BDMR3FDMQ1QRL9JAEJ"
    # (5 unchanged attributes hidden)
}

Plan: 0 to add, 2 to change, 0 to destroy.

```

Je fais un terraform plan, et on voit bien que c'est la preuve que mes variables fonctionnent bien. Le plan est identique à avant, juste avec des valeurs qui viennent maintenant de terraform.tfvars au lieu d'être en dur.

Je fais un terraform apply :

```

vmworkstation_vm.victim: Modifying... [id=59TLE58MELTTL3BDMR3FDMQ1QRL9JAEJ]
vmworkstation_vm.kali_like: Modifying... [id=QGFAB6PDLF5P09TVICA5VPJA14KMIPM7]
vmworkstation_vm.victim: Modifications complete after 0s [id=59TLE58MELTTL3BDMR3FDMQ1QRL9JAEJ]
vmworkstation_vm.kali_like: Modifications complete after 0s [id=QGFAB6PDLF5P09TVICA5VPJA14KMIPM7]

Apply complete! Resources: 0 added, 2 changed, 0 destroyed.
PS C:\Users\lala\terraform_debian_lab>

```

Toute la partie technique de PDF est bouclée !